

Separar la Lógica de Negocio de la Persistencia

Problemas de la capa service actual

```
...
private static final String ADDMECHANIC =
    "insert into TMechanics...";
...
public MechanicDto execute()
    throws BusinessException {
    ...
    c = Jdbc.getConnection(...);
    pst = c.prepareStatement(ADDMECHANIC);
    pst.setString(1, mechanic.id);
    pst.setString(2, mechanic.nif);
    ...
    pst.executeUpdate();
    ...
}
```

```
URL = ConnectionProperties
    .getProperty("DB_URL");
USER = ConnectionProperties
    .getProperty("DB_USER");
PASS = ConnectionProperties
    .getProperty("DB_PASS");

Connection createThreadConnection()
    throws SQLException {
    Connection con = DriverManager
        .getConnection(URL, USER, PASS);
    threadConnection.set(con);
    return con;
}
```

¿ Qué ocurre si ...

- cambia la tecnología?
- se añade pool de conexiones?
- cambian los datos de conexión de la bbdd?
- cambia el nombre de la tabla o de las columnas?

Dividir la capa de negocio: Capa de negocio + Capa de acceso a datos

Separación en capas

Lógica de negocio **service**

Acceso a datos **persistence**

Patrones nuevos para la capa de acceso a datos

- Patrón DAO (Core J2EE Patterns) [7]. Interfaz de acceso a datos, independiente de la tecnología.
- Table Data Gateway (Fowler), posible implementación del anterior basada en tablas.

Patrones nuevos para la capa de lógica de negocio

- Transaction scripts
- Command

Separar la lógica de negocio del acceso a datos:
Persistencia

Patrones para la capa de acceso a datos

Martin Fowler, Patterns of Enterprise Application Architecture

<http://martinfowler.com/eaCatalog/index.html>

Data Source Architectural Patterns

Table Data Gateway

An object that acts as a gateway to a database table. One instance handles all the rows in the table.

Row Data Gateway

An object that acts as a gateway to a single record in a data source. There is one instance per row.

Active Record

An object that wraps a row in a database table or view, encapsulates the database access, and adds domain logic on that data.

Data Mapper

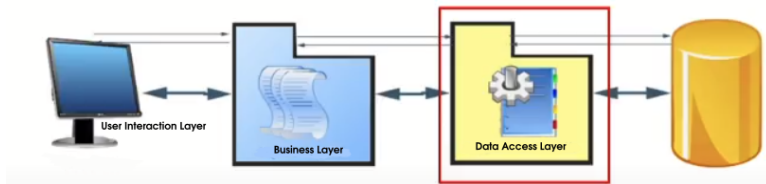
A layer of mappers that moves data between objects and a database while keeping them independent of each other and the mapper itself.

J2EE Patterns <https://www.oracle.com/java/technologies/dataaccessobject.html>

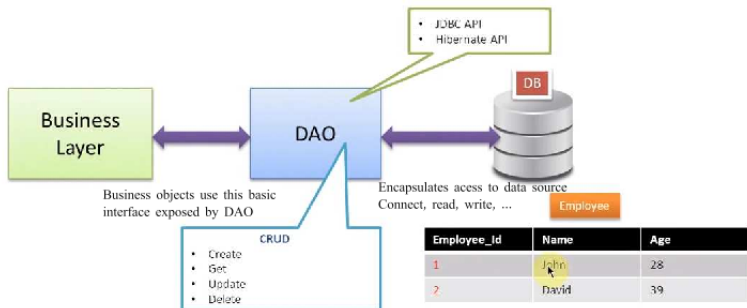
Patrón DAO (Data Access Object)

El patrón **DAO** [10] separa la lógica de negocio del acceso a los datos. La aplicación trabaja con objetos DAO, y son estos los que se encargan de comunicarse con la fuente de datos.

La capa de negocio no necesita saber cómo se accede a la base de datos.



Responsabilidades de los objetos DAO



- Abstraen y centralizan el **acceso a la fuente de datos**.
- Ofrecen una interfaz con **operaciones de acceso y manipulación de datos** independiente de su implementación.
- Transforman los datos obtenidos en **objetos Java**.
- Desacoplan la capa service de los detalles de **almacenamiento y recuperación**.

Diagrama de clases

Cliente: necesita acceso a la fuente de datos.

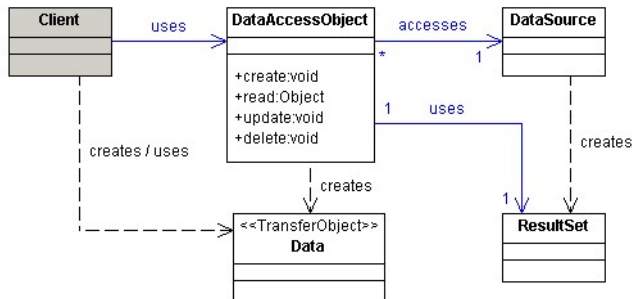
DAO: separa el acceso a datos de la fuente de datos.

DataSource: la fuente de datos.

Resultado de la consulta: los datos obtenidos de la fuente.

ResultSet en una implementación JDBC.

Transfer Object: la información que se devuelve a la aplicación.



Principales ventajas del patrón DAO

1. Separación de responsabilidades

- La lógica de negocio no necesita conocer los detalles del acceso a los datos.
- El DAO centraliza las operaciones de persistencia.

2. Desacoplamiento y facilidad de mantenimiento

- Permite modificar la implementación del acceso a datos sin afectar a la lógica de negocio.
- Por ejemplo, cambiar la tecnología de persistencia manteniendo la misma interfaz.

3. Reutilización y centralización

- Las operaciones de acceso a datos se concentran en un único componente.
- Evita duplicar código en diferentes partes de la aplicación.

4. Facilita las pruebas unitarias

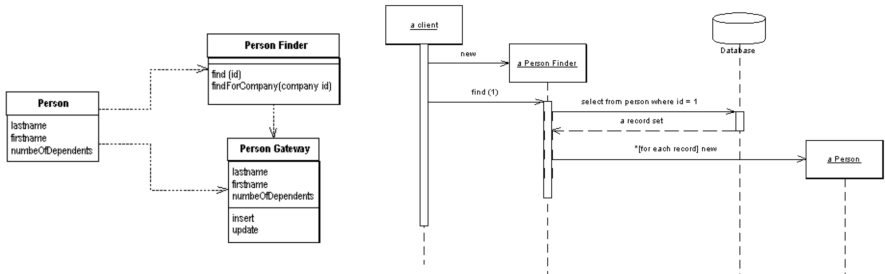
- Se pueden utilizar implementaciones simuladas (*mock*) del DAO.
- Permite probar la lógica de negocio sin depender de una base de datos real.

Posibles implementaciones del patrón DAO

Row Data Gateway

- Un objeto Row Data Gateway [12] representa **una única fila de la tabla** y cada fila de la tabla se representa por **exactamente** un objeto en memoria.
- Cada atributo corresponde a una columna de la tabla.
- Sus métodos permiten crear, actualizar o eliminar **esa fila concreta**.
- Un **Finder** se encarga de ejecutar consultas y crear los objetos Row Data Gateway correspondientes.
- No contiene lógica de negocio, solo código de acceso y persistencia.

Row Data Gateway



MechanicGateway

Row Data Gateway en CWS

```
public class MechanicGateway {
    private final String id; // read-only!
    private long version;
    private String nif;
    ...

    public MechanicGateway(String nif, ...) {
        this.id = UUID.randomUUID();
        ...
        this.version = 1L;
    }

    public void add() {
        try (Connection c = ...;
            PreparedStatement pst = ... {
            pst.setString(1, id);
            ...
            pst.executeUpdate();
        } catch (SQLException e) {
            ....
        }

    public void delete() {
        try (Connection c =
            PreparedStatement ps = {
            ps.setString(1, id);
            ps.executeUpdate();
        }
    }
}
```

13 of 46

```
public void update() {
    try (Connection c = ...;
        PreparedStatement ps = ...) {
        ps.setString(1, nif);
        ps.setString(2, name);
        ps.executeUpdate();
    }
}

class FindMechanic {
    private String findAll =
        "SELECT * from TMechanics ...";

    public MechanicGateway findById(int id) {
        try ( Connection c = ...;
            PreparedStatement pst = ...;
            ResultSet rs = pst.executeQuery(); ) {
            if (rs.next()) {
                return new MechanicGateway(rs.getString(),
                    rs.getString(),
                    ...);
            }
            return result;
        } catch (SQLException ex) {
            throw new RuntimeException(ex.Message);
        }
    }
}
```

Row Data Gateway: Ventajas e inconvenientes

Ventajas

- Centrado en el acceso a datos.
- Estructura similar a la tabla.
- Fácil de implementar en dominios de problema sencillos.

Inconvenientes

- Fuerte acoplamiento entre objetos y esquema de la BD.
- Difícil de escalar en dominios complejos.
 - La capa de servicio debe conocer cómo están organizados los datos en la base de datos (qué tablas hay, cómo se relacionan, qué campos se necesitan)
 - Ejemplo: Borrar mecánico.

Patrón Table Data Gateway

- El patrón **Table Data Gateway** [9] organiza el acceso a los datos mediante una clase asociada a **una tabla o vista**, concentrando sus operaciones de consulta y modificación.
- Expone una **interfaz sencilla** con
 - un **único método** para cada operación de **inserción, actualización y eliminación** de filas de **esa tabla**.
 - **varios métodos** para **consultar** filas de esa tabla, incluso mediante consultas complejas (encontrar mecánicos con intervenciones pendientes).
- Cada método recibe parámetros, ejecuta la sentencia SQL correspondiente y puede devolver algún resultado utilizando el patrón DTO.
- No contiene lógica de negocio: su única responsabilidad es **acceder a los datos de la tabla**.

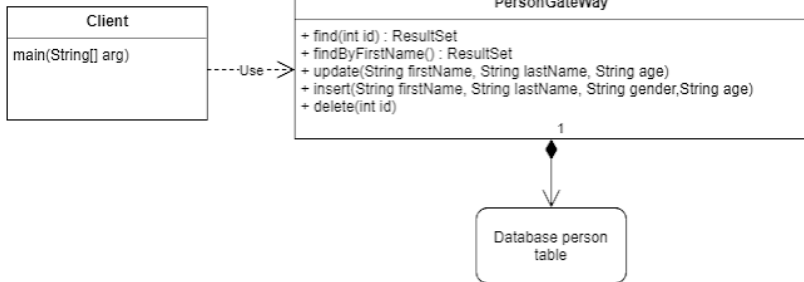
Diferencia DAO - TDG

DAO

- Organiza el acceso en torno a conceptos del dominio
- (MechanicDao, WorkOrderDao, InvoiceDao)

TDG

- Organiza el acceso en torno a tablas de la BD
- (MechanicsGateway, WorkOrdersGateway, InvoicesGateway)



Ventajas e inconvenientes del patrón TDG

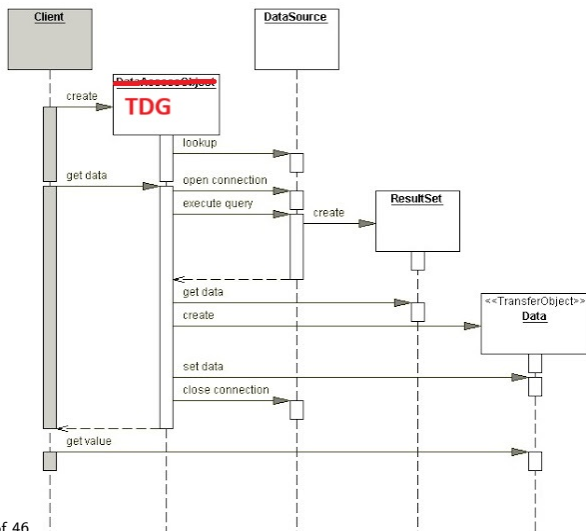
Ventajas

- Centraliza el acceso a la base de datos, facilitando el mantenimiento y la evolución del código
- Interfaz sencilla para operaciones sobre la tabla
- Implementación sencilla que acelera el desarrollo inicial

Inconvenientes

- Muy acoplado al esquema de la base de datos
- Puede crecer demasiado si concentra demasiadas consultas
- Escalabilidad limitada, haciendo que la lógica de aplicación conozca detalles de persistencia.

Diagrama de secuencia



Ejercicio

1. Identificar los participantes del patrón TDG
A partir del diagrama del patrón DAO, identifica en el código proporcionado los elementos que podrían corresponder a Client, Data Access Object, DataSource, Transfer Object
2. En el código de DeleteMechanic y UpdateMechanic, identifica las instrucciones que realizan directamente operaciones de acceso a la base de datos.
Indica qué responsabilidad debería trasladarse a la capa de acceso a datos.
3. Suponiendo que utilizamos un Table Data Gateway (TDG) por tabla, determina qué TDG son necesarios para resolver las operaciones realizadas por UpdateMechanic y DeleteMechanic. Para cada TDG, indica:
 - ☐ qué tabla representa;
 - ☐ qué métodos debe proporcionar;
 - ☐ qué parámetros recibe cada método;

Ejercicios (1)

1. **Identificación del patrón.** A partir del diagrama anterior, identifica en el código anexo los elementos que podrían corresponder a Client, Data Access Object, DataSource y Data Transfer Object. Justifica tus respuestas.
2. **Responsabilidades.** Identifica las instrucciones de UpdateMechanic y DeleteMechanic que dependen de la tecnología utilizada para el acceso a la base de datos. ¿Qué problemas plantea que estas operaciones estén implementadas directamente en estas clases?
3. **Diseño de los TDG.** Suponiendo que utilizamos un Table Data Gateway por tabla, determina qué TDG son necesarios. Para cada uno, indica los métodos, parámetros y valores de retorno necesarios.

Ejercicios (2)

4. **Implementación de MechanicGateway.** Implementa los métodos necesarios de MechanicGateway para reemplazar las operaciones de acceso a datos de UpdateMechanic y DeleteMechanic.
5. **DTO.** ¿Qué objetos de transferencia son necesarios para implementar estas operaciones? ¿Pueden reutilizarse los DTO existentes en la capa de servicio? Justifica la respuesta. Si consideras necesario introducir nuevos DTO, propón su diseño.
6. **Refactorización.** Reescribe DeleteMechanic utilizando los gateways anteriores. La clase debe conservar las reglas de negocio, pero no debe realizar directamente operaciones sobre la base de datos.

Ejercicios (3)

7. **Análisis del acoplamiento.** Compara la implementación original y la refactorizada. ¿Qué dependencias con la tecnología de persistencia tenía DeleteMechanic? ¿Cuáles conserva después de aplicar DAO/TDG? ¿Qué ocurriría si mañana cambiásemos JDBC por otra tecnología de persistencia? ¿Qué clases tendrían que modificarse y cuáles no?

```

public interface Gateway<T> {
    /**
     * Adds a new item to the table
     * @param new item
     */
    void add(T t) throws PersistenceException;

    /**
     * Removes an object from the table
     * @param key to delete
     */
    void remove(String id) throws PersistenceException;

    /**
     * Updates a row
     * @param new data to overwrite old one
     */
    void update(T t) throws PersistenceException;

    /**
     * Finds a row in the table
     * @param record's primary key to retrieve
     * @return dto from that record, probably null
     */
    Optional<T> findById(String id) throws PersistenceException;

    /**
     * Retrieves all data in a table
     */
    List<T> findAll() throws PersistenceException;
}

```

```

public interface MechanicGateway
extends Gateway<MechanicRecord> {
    /**
     * @param nif
     * @return the mechanic record with
     * the nif or empty if none
     */
    Optional<MechanicRecord>
    findByNif(String nif)
    throws PersistenceException;

    public static class MechanicRecord {
        public String id;
        public Long version;
        public LocalDateTime createdAt;
        public LocalDateTime updatedAt;
        public String entityState;

        public String nif;
        public String name;
        public String surname;
    }
}

```

El DTO es una copia del contenido de la base de datos

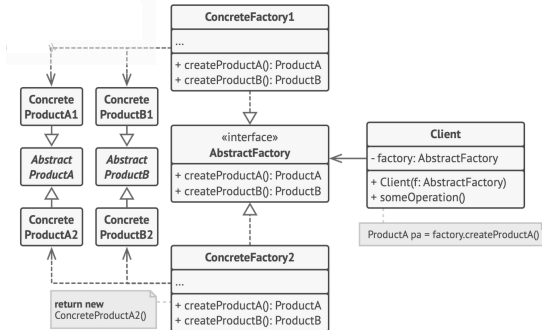
¿ Cómo se crean los TDG?

Con frecuencia, se necesitan varios TDG (MechanicGateway, WorkOrderGateway, ...)

- **Estrategia de generación automática del código TDG**
- **Considera usar algún patrón factoría para incrementar la flexibilidad**
 1. **Abstract Factory**
 2. **Factory Method**
 3. **Class Factory** o **Simple Factory**: Opción preferida.

Abstract Factory

- Patrón de diseño creacional que proporciona una interfaz para crear familias de objetos relacionados sin especificar sus clases concretas.
- Útil si la base de datos puede cambiar de tecnología o de proveedor.
- Acoplamiento concentrado: el cliente sufre cambios mínimos en su código.



Patrón TDG y Abstract Factory

Sin Abstract Factory

```
OracleMechanicGateway m = new OracleMechanicGatewayImpl();  
OracleInterventionGateway i = new OracleInterventionGatewayImpl();
```



```
HsqlMechanicGateway m = new HsqlMechanicGatewayImpl();  
HsqlInterventionGateway i = new HsqlInterventionGatewayImpl();
```

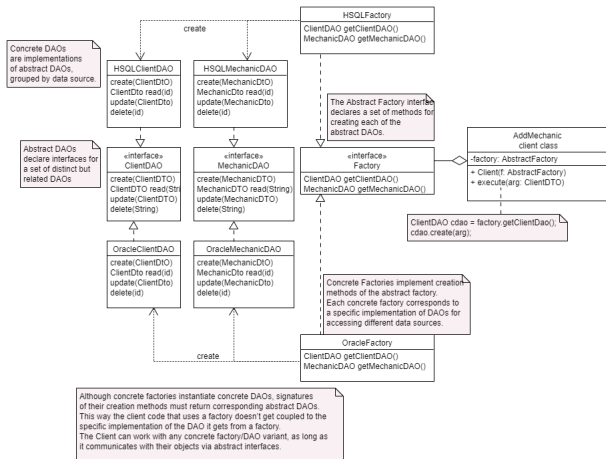
Con Abstract Factory

```
GatewayFactory factory = new OracleFactory();  
Gateway m = factory.createMechanicGateway();  
Gateway i = factory.createInterventionGateway();
```



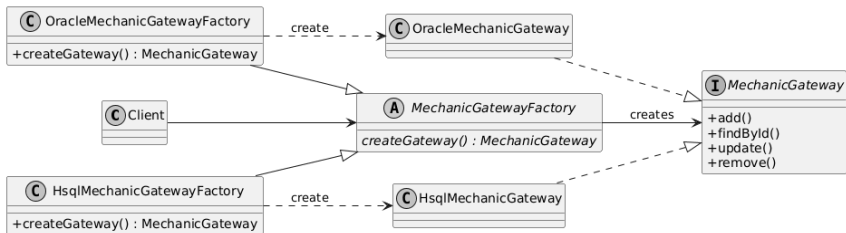
```
GatewayFactory factory = new HsqlFactory();  
Gateway m = factory.createMechanicGateway();  
Gateway i = factory.createInterventionGateway();
```

Patrón TDG y Abstract Factory

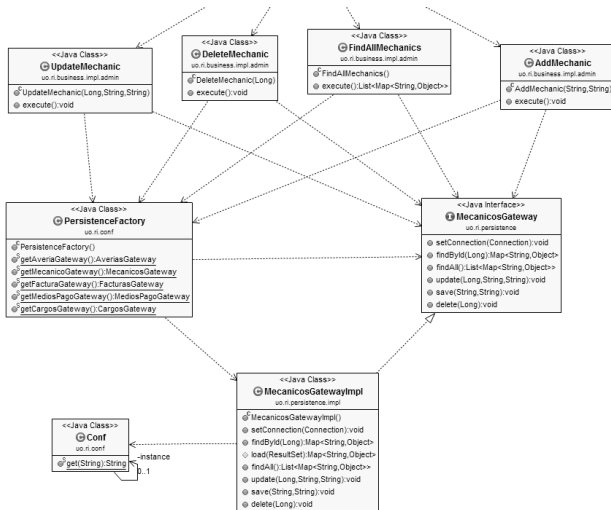


Factory Method

Patrón de diseño creacional que define una interfaz para crear un objeto, pero deja que las subclases decidan qué clase concreta instanciar.



Patrón TDG y Simple Factory



Separar la Lógica de Negocio de la Persistencia: Negocio

Patrones para la capa de negocio (service)

¿Cómo podemos organizar el código dentro de la capa de negocio?

Hay varios posibles patrones:

- **Transaction Scripts** el más simple.
- Domain Model
- Table Module

Transaction Scripts

Patrón que organiza la lógica de negocio en procedimientos o *scripts*, donde cada uno implementa una **operación de negocio completa** ejecutada de forma **transaccional**.

- Recibe los datos necesarios para la operación.
- Valida y procesa los datos.
- Accede a la base de datos mediante patrones de acceso a datos.
- Devuelve el resultado de la operación.

Los scripts pueden organizarse:

- Agrupados en una misma clase; cada script es un procedimiento.
- **Cada script en su propia clase**, por ejemplo mediante el patrón **Command**.

Ventajas y desventajas de Transaction Script

Ventajas:

- Ideal para sistemas con poca lógica de negocio.
- Modelo procedimental simple y fácil de entender.
- Baja sobrecarga de rendimiento.
- Funciona bien con una capa de datos simple (p.ej., Row Data Gateway, Table Data Gateway).

Desventajas:

- Difícil asegurar que la invocación sea transaccional; riesgo de duplicación de código en transacciones.
- No apto para modelos de dominio complejos.

Ejercicios

Considera la gestión de mecánicos.

- ¿ Cuáles son nuestros Transaction Scripts ?
- ¿ Qué es necesario para que la ejecución de los TS sea transaccional?. Ver página siguiente.
 - ☐ `conn.setAutoCommit(boolean autoCommit)`
 - ☐ `conn.commit()`
 - ☐ `conn.rollback()`

```

class AddMechanic{
    private MechanicGateway mg = ...;
    private MechanicDto dto;
    public AddMechanic(
        MechanicDto arg) {
        ...
    }

    public MechanicBLDto execute()
        throws ... {
        checkNotExist();
        // mg.findByNif();
        checkCanBeDeleted();
        // connect, exec
        insertMechanic();
        // mg.add(m);
        return dto;
    }
}

class MechanicGateway {
    public Optional<MechanicRecord>
        findByNif(nif) {
        // connect
        // exec query
        // return dto
    }

    public void add(MechanicRecord m)
        // connect
        // exec insert
    }
}

```

Patrón Command

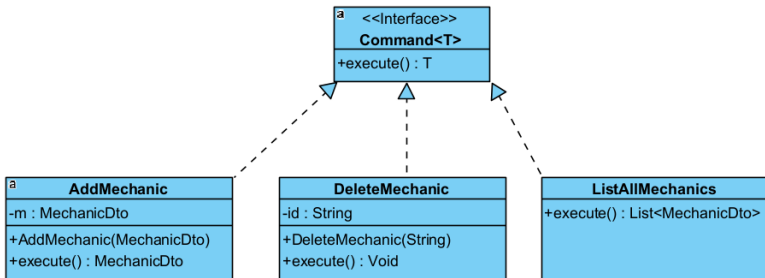
Los Transaction Scripts tienen una estructura similar y **repiten código de gestión de transacciones**:

- Obtener una conexión.
- Desactivar el *autocommit*.
- Ejecutar las operaciones.
- Hacer *commit*.
- Hacer *rollback* si error.

```
class AddMechanic {  
    public MechanicDto execute() {  
        try(Connection c = ...) {  
            c.setAutoCommit(false);  
            gateway.findByNif();  
            gateway.add();  
            c.commit();  
        } catch (SQLException e) {  
            c.rollback();  
            ...  
        }  
        return dto;  
    }  
}
```

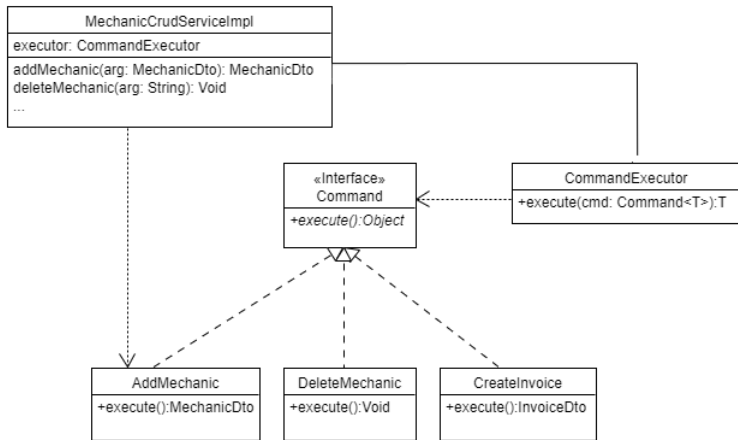
El patrón **Command** permite encapsular cada operación de negocio (Transaction Script) en un objeto y delegar la ejecución del **código común**, evitando su repetición.

- Cada operación se encapsula en un **objeto Command**.
- Las operaciones se ejecutan mediante una interfaz común.
- La **gestión de la transacción** se centraliza.



Clases principales

- **Command**: interfaz con un método común `execute()`.
- **Comandos concretos**: implementan `execute()` con la lógica de cada operación de negocio.
- **CommandExecutor**: recibe un `Command` y lo ejecuta.
 - Centraliza la lógica común: **transacciones**, control de errores, logs, etc.
- **Cliente**: crea el `Command` y lo pasa al `CommandExecutor`.



```
public class CommandExecutor {  
    public <T> T execute(Command<T> cmd) throws BusinessException {  
        try (Connection c = Jdbc.createThreadConnection()){  
            try {  
                c.setAutoCommit(false);  
                T res = cmd.execute();  
                c.commit();  
                return res;  
            } catch (BusinessException e) {  
                c.rollback();  
                throw e;  
            }  
        } catch (SQLException | PersistenceException e) {  
            throw new RuntimeException(e);  
        }  
    }  
}
```

Ventajas

- **Desacopla** las clases que invocan la operación de la que sabe cómo llevarla a cabo (AddMechanic in CWS1).
- **Extensibilidad** Añadir un nuevo command se puede hacer sin cambiar el código existente.

Ejercicio

Suponiendo el CommandExecutor de la página anterior:

- Reescribe un TS
- Reescribe el código de la fachada que llama a ese TS.

Further Reading (1)

- [1] <https://www.youtube.com/watch?v=8Zoz2njk5f8>
- [2] <https://www.oreilly.com/library/view/software-architecture-patterns/9781491971437/ch01.html>
- [3] <https://javadevguy.wordpress.com/2019/01/06/reevaluating-the-layered-architecture/>
- [4] <https://martinfowler.com/bliki/PresentationDomainDataLayering.html>
- [5] <https://refactoring.guru/design-patterns/facade>
- [6] <https://refactoring.guru/design-patterns/factory-comparison>

Further Reading (2)

- [7] <https://www.oracle.com/java/technologies/data-access-object.html>
- [8] <https://www.martinfowler.com/eaCatalog/tableDataGateway.html>
- [9] <https://www.sourcecodeexamples.net/2018/04/table-data-gateway.html>
- [10] <https://www.oracle.com/technetwork/java/dataaccessobject-138824.html>
- [11] <https://martinfowler.com/eaCatalog/gateway.html>
- [12] <https://www.sourcecodeexamples.net/2018/04/row-data-gateway.html>

Further Reading (3)

- [13] <https://martinfowler.com/eaCatalog/gateway.html>
- [14] POJOs in Action: Developing Enterprise Applications with Lightweight Frameworks. Chris Richardson. ISBN 1932394583. Manning Publications Co. 2006.